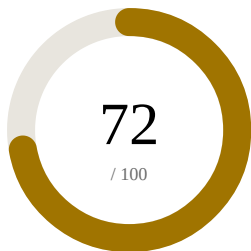


Sample Co (PG)

Engine: **PostgreSQL** 15.18 (Debian 15.18-1.pgdg13+1) · Collected: 05 Jul 2026 13:01 UTC · Database size: 1.0 GB



Health score: 72

queries -20, indexes -8

Executive summary

- Overall health score: 72/100 — degraded; the issues below cost real performance.
- 10 findings in total: 1 critical, 3 medium, 4 low, 2 info.
- Biggest issue: one query consumes a large share of total database time (SELECT id, status, total_cents FROM orders WHERE customer_id = ?).
- 4 index findings — usually the cheapest wins in this report.
- 2 configuration settings worth reviewing; each cites its current value and a recommended range.

Top 5 issues

CRITICAL R-Q1

1. One query consumes a large share of total database time

```
SELECT id, status, total_cents FROM orders WHERE customer_id = ?
```

calls	3108
total_ms	177949.4
pct_of_db_time	54.6
mean_ms	57.26
rows_per_call	2.1
query_digest	4378614815602751461

This single statement dominates database time. Investigate its execution plan first; typical fixes are an index on its filter columns, caching its result if it repeats, or restructuring the query. Test any change in staging first.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

MEDIUM R-I1

2. Table is scanned in full by frequent queries (missing index candidate)

orders

table_size	177.4 MB
candidate_columns	customer_id
seq_scans	9684

Queries filter orders on (customer_id) without a usable index, so the table is read in full each time. Consider an index on (customer_id). Test in staging first: measure the query with EXPLAIN before and after.

⚠ Confidence: medium — test this change in staging before applying it to production.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

MEDIUM R-Q3

3. Query runs at very high frequency

SELECT product_id, quantity, price_cents FROM order_items WHERE order_id = ?

calls_since_stats_reset	64560.0
calls	64560
mean_ms	0.01
total_ms	525.3
query_digest	5026988254128095876

This statement runs extremely often. Check whether the application repeats it per item instead of batching (N+1 pattern), and whether its result can be cached. Even a fast query at this volume costs real capacity.

⚠ Confidence: low — test this change in staging before applying it to production.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

MEDIUM R-I3

4. Redundant index duplicates another index's leading columns

public.users.users_email_idx

redundant_index	CREATE INDEX users_email_idx ON public.users USING btree (email)
covered_by	users_email_dup_idx

Index users_email_idx is covered by another index with the same leading columns; it adds write cost and space for no read benefit. Verify with usage stats over a full business cycle, then drop the redundant one. Test in staging first: measure the query with EXPLAIN before and after.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

5. Index is never used but costs space and write time

public.users.users_email_dup_idx

scans_since_stats_reset	0
index_size	68.0 MB
definition	CREATE INDEX users_email_dup_idx ON public.users USING btree (email)

Index users_email_dup_idx shows no reads but is maintained on every write. Verify it is not needed by replicas, rare reports, or seasonal jobs, then drop it. Keep the definition somewhere so it can be recreated.

⚠ Confidence: medium — test this change in staging before applying it to production.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

Slow queries

Statement (normalized)	Calls	Total ms	Mean ms	% of DB time
SELECT id, status, total_cents FROM orders WHERE customer_id = ?	3,108	177,949	57.26	54.6
SELECT pg_sleep(?)	6	48,046	8007.66	14.7
SELECT id, user_id, kind FROM events ORDER BY id LIMIT ? OFFSET ?	3,228	47,535	14.73	14.6
UPDATE users SET name = ? WHERE id = ?	12	42,051	3504.23	12.9
SELECT id, status, total_cents FROM orders WHERE customer_id = ?	120	6,759	56.33	2.1
SELECT id, email FROM users WHERE id = ?	31,226	2,171	0.07	0.7
ANALYZE	1	857	856.51	0.3
SELECT product_id, quantity, price_cents FROM order_items WHERE order_id = ?	64,560	525	0.01	0.2
SELECT id FROM orders WHERE id > ? ORDER BY id LIMIT ?	3,228	128	0.04	0.0
SELECT id, email FROM users WHERE id = ?	1,054	73	0.07	0.0

Top statements by cumulative execution time since statistics were last reset. All literals are stripped before data leaves your environment.

Index advisor

Severity	Finding	Object	Evidence
MEDIUM	Table is scanned in full by frequent queries (missing index candidate)	orders	table_size=177.4 MB; candidate_columns=customer_id; seq_scans=9684
MEDIUM	Redundant index duplicates another index's leading columns	public.users.users_email_idx	redundant_index=CREATE INDEX users_email_idx ON public.users USING btree (email); covered_by=users_email_dup_idx
LOW	Index is never used but costs space and write time	public.users.users_email_dup_idx	scans_since_stats_reset=0; index_size=68.0 MB; definition=CREATE INDEX users_email_dup_idx ON public.users USING btree (email)
LOW	Index is never used but costs	public.users.users_email_idx	scans_since_stats_reset=0; index_size=68.0 MB; definition=CREATE INDEX

space and write
time

users_email_idx ON public.users USING
btree (email)

Growth & operational risk

Connections at collection time: **6** of **100** allowed.

No lock, connection, or growth risks detected on this run.

Vacuum health (PostgreSQL)

Table	Size	Dead tuples %	Last autovacuum
public.order_items	423.7 MB	0.0	never
public.users	237.9 MB	0.0	never
public.events	223.8 MB	0.0	never
public.orders	177.4 MB	0.0	never

Configuration findings

INFO effective_cache_size

effective_cache_size 4 GB (compiled default)

effective_cache_size is still the compiled default (4 GB). It costs nothing at runtime — it only tells the planner how much cache the OS likely has. Left too low, the planner avoids indexes it should use. Suggested change: Set it to 50-75% of the machine's RAM; reload is enough.

INFO random_page_cost

random_page_cost 4.0

random_page_cost=4 models spinning disks. On SSD/NVMe storage random reads are nearly as cheap as sequential — with the old value the planner avoids index scans it should choose. Suggested change: If storage is SSD, set random_page_cost between 1.1 and 1.5; reload is enough.

Developer action plan

- [Top issue 1] One query consumes a large share of total database time — SELECT id, status, total_cents FROM orders WHERE customer_id = ?. This single statement dominates database time.

2. [Top issue 2] Table is scanned in full by frequent queries (missing index candidate) — orders. Queries filter orders on (customer_id) without a usable index, so the table is read in full each time.
3. [Top issue 3] Query runs at very high frequency — SELECT product_id, quantity, price_cents FROM order_items WHERE order_id = ?. This statement runs extremely often.
4. [Top issue 4] Redundant index duplicates another index's leading columns — public.users.users_email_idx. Index users_email_idx is covered by another index with the same leading columns; it adds write cost and space for no read benefit.
5. [Top issue 5] Index is never used but costs space and write time — public.users.users_email_dup_idx. Index users_email_dup_idx shows no reads but is maintained on every write.
6. Re-run the collector after each change to verify the effect before the next one.

Every recommendation above should be validated in a staging environment first. Capabilities on this run: table_stats, index_stats, lock_waits, sessions, config, query_stats, io_timing.